AGENTIC CODING CLAUDE CODE V



Build Production-Grade Enterprise Apps

AGENDA

Test Pyramid

Apply the test pyramid: many fast unit tests, fewer integration tests, and a relatively small number of E2E tests.

Test-Driven Development (TDD)

Apply the TDD Red-Green-Refactor cycle by adding a TDD governance rule to CLAUDE.md. Define the test commands explicitly in package.json.

TDD Unit Tests

Place the TDD governance rule in the project's CLAUDE.md. Add tests/unit/test_name.test.ts to the current project and run the test command bun run test:unit.

Integration Tests

Run integration tests for all repository and service operations by adding this testing rule to CLAUDE.md. Run an integration test using the test command bun run test:integration.

Hooks for Deterministic Integration Tests

To trigger unit and integration tests deterministically before every Git commit, add a repository-managed hook at .githooks/pre-commit and configure Git to invoke the hook automatically.

CI for Deterministic Integration Tests

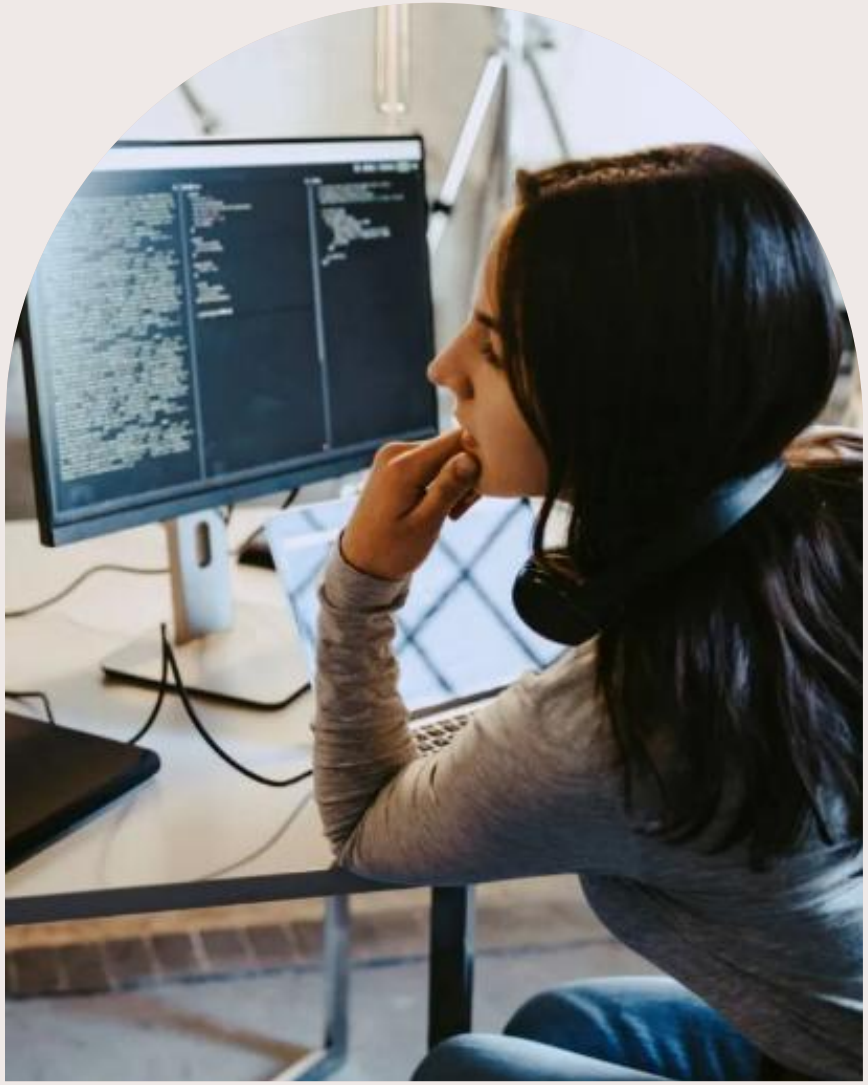
To trigger unit and integration tests deterministically in CI (Continuous Integration), create a GitHub Actions workflow in the integration-tests.yml file.

End-to-End (E2E) UI Test

Install the Playwright E2E test automation framework and configure it in playwright.config.ts. Run E2E tests using Playwright, seeing the browser live during the test.

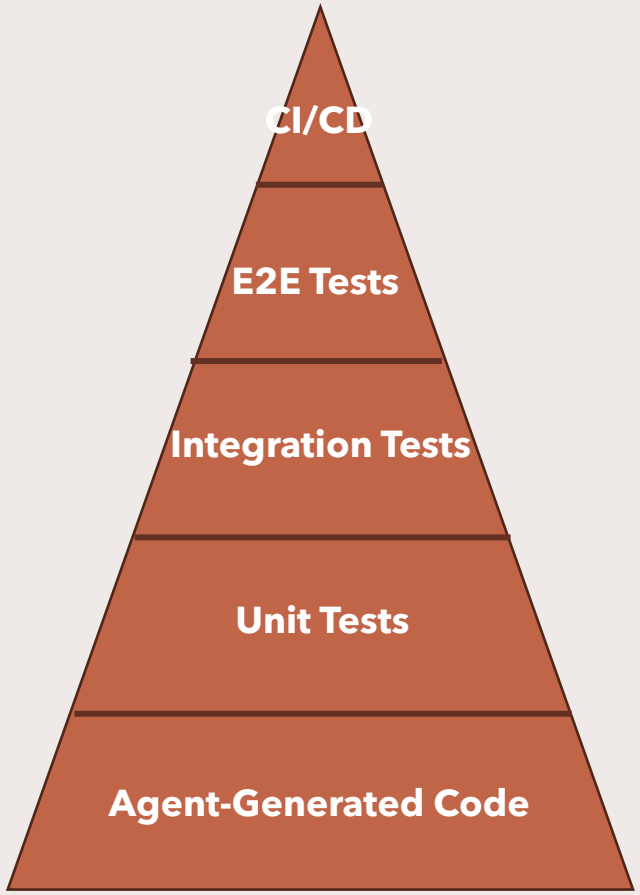
Quick Reference

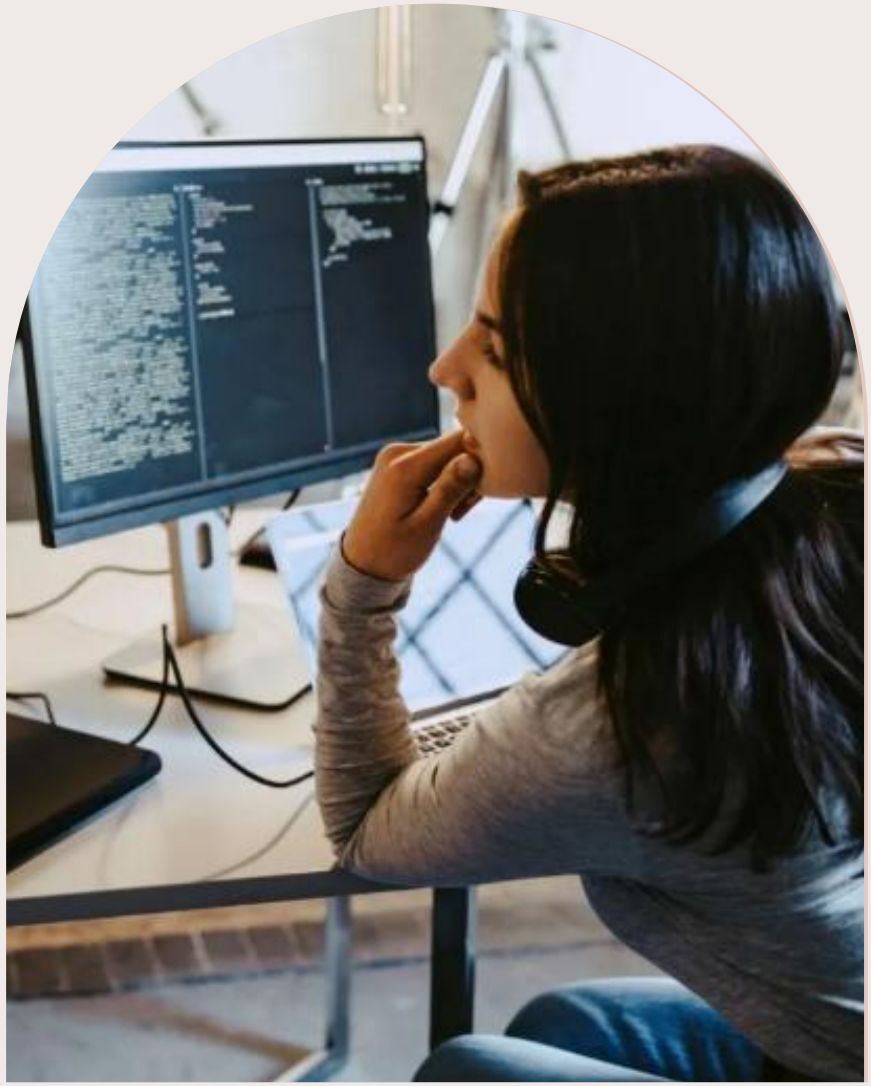
Review of commands, paths, and keyboard combinations discussed in this session.



TEST PYRAMID

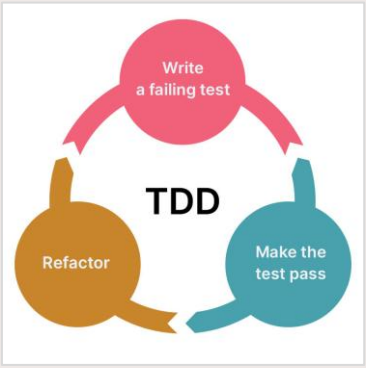
- One of the best practices of testing is the **test pyramid**: many fast unit tests, fewer integration tests, and a relatively small number of E2E tests.
- Unit tests are fast, inexpensive to run, precise when they fail, good for edge cases, and easy to run after every small code change. They can check if each important piece of logic behaves correctly, including edge cases.
- Integration tests verify boundaries where defects frequently occur. They can check if the application layers and external components work together correctly.





Test-Driven Development (TDD)

- **One of the best practices of unit testing is Test-Driven Development (TDD), which runs the red-green-refactor cycle with each unit test case:**
  1. **Write a unit test.**
  2. **Run the unit test.**
  3. **Test fails. (Red)**
  4. **Implement minimum code.**
  5. **Run the unit test.**
  6. **Test passes. (Green) Otherwise, go back to 4.**
  7. **Refactor the code. (Refactor)**
  8. **Run the unit test.**
  9. **Test passes. Otherwise, go back to 7.**



TDD UNIT TESTS

CLAUDE.md > # CLAUDE.md

```
# CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

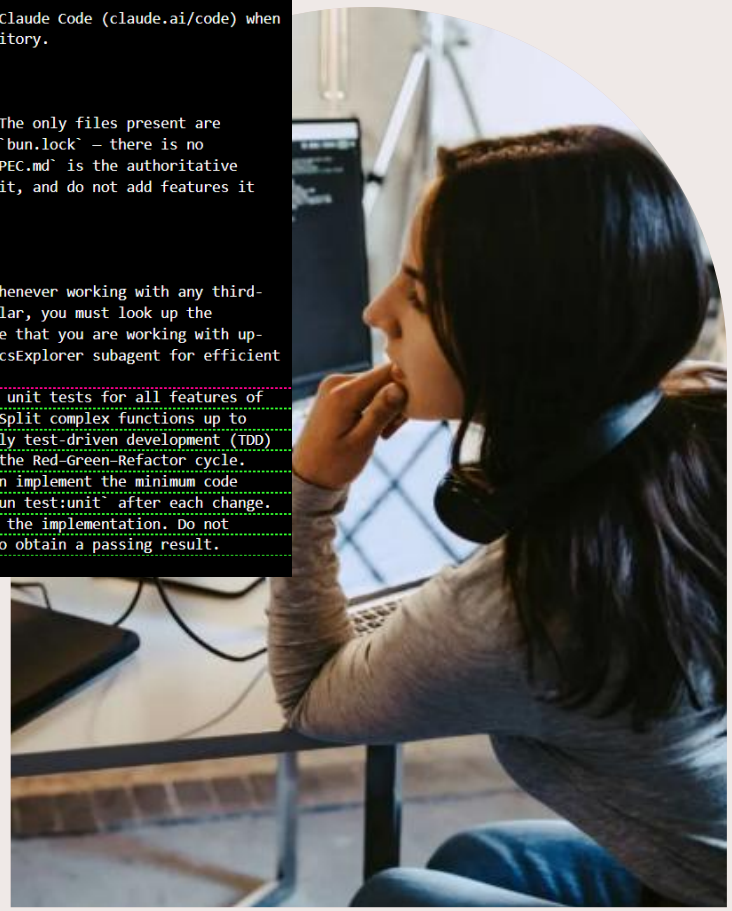
## Status

This project is **spec-only**. The only files present are `SPEC.md`, `package.json`, and `bun.lock` — there is no application source code yet. `SPEC.md` is the authoritative design document: build against it, and do not add features it excludes.

## Policy

- **Documentation Accuracy.** Whenever working with any third-party library or something similar, you must look up the official documentation to ensure that you are working with up-to-date information. Use the DocsExplorer subagent for efficient documentation lookup.
- **Testing Expectations.** Add unit tests for all features of this app. Add mocks as needed. Split complex functions up to simplify testing if needed. Apply test-driven development (TDD) in writing code; i.e., execute the Red-Green-Refactor cycle. First write the unit tests, then implement the minimum code needed to pass them. Run `bun run test:unit` after each change. If tests fail, diagnose and fix the implementation. Do not delete, weaken, or skip tests to obtain a passing result.

```



- Open the **note-taking** folder in the VSCode extension so that VSCode reloads with **note-taking** as the workspace root and so the Claude Code panel treats it as the project.
- Run **/clear**.
- Define the test commands explicitly in **package.json**:
  - ```
    {
      "scripts": {
        "test:unit": "bun test tests/unit",
        "test:integration": "bun test tests/integration",
        "test:e2e": "playwright test"
      }
    }
    ```
- Place the following governance rule in the project's **CLAUDE.md**:
  - **Add unit tests for all the features of this app. Add mocks as needed. Split complex functions up to simplify testing if needed. Apply test-driven development (TDD) in writing code; i.e., execute the Red-Green-Refactor cycle. First write the unit tests, then implement the minimum code needed to pass them. Run "bun run test:unit" after each change. If tests fail, diagnose and fix the implementation. Do not delete, weaken, or skip tests to obtain a passing result.**
    ```

TDD UNIT TESTS

tests > unit > TS validation.test.ts > ...

```ts
import { describe, expect, it } from 'bun:test';

import { noteInputSchema } from '@/lib/validation';

const validDoc = JSON.stringify({
  type: 'doc',
  content: [{ type: 'paragraph', content: [{ type: 'text', text: 'hello' }] }],
});

describe('noteInputSchema', () => {
  it('accepts valid title/content and defaults isPublic to false', () => {
    const parsed = noteInputSchema.parse({
      title: '  My Note  ',
      contentJson: validDoc,
    });

    expect(parsed.title).toBe('My Note');
    expect(parsed.contentJson).toBe(validDoc);
    expect(parsed.isPublic).toBe(false);
  });

  it('rejects invalid content json payload', () => {
    const result = noteInputSchema.safeParse({
      title: 'Title',
      contentJson: '{"type":"paragraph"}',
    });

    expect(result.success).toBe(false);
  });

  it('rejects titles over 100 chars', () => {
    const result = noteInputSchema.safeParse({
      title: 'a'.repeat(101),
      contentJson: validDoc,
    });

    expect(result.success).toBe(false);
  });
});

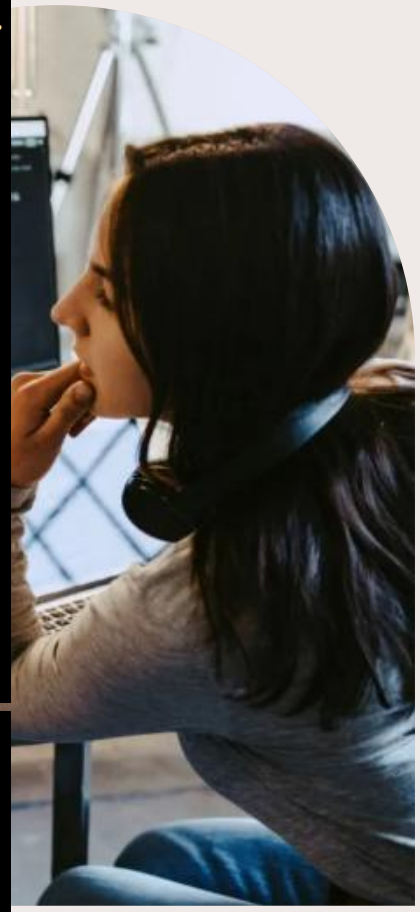
```

Running the unit test command now and I'll report the result directly.

Ran `bun run test:unit`

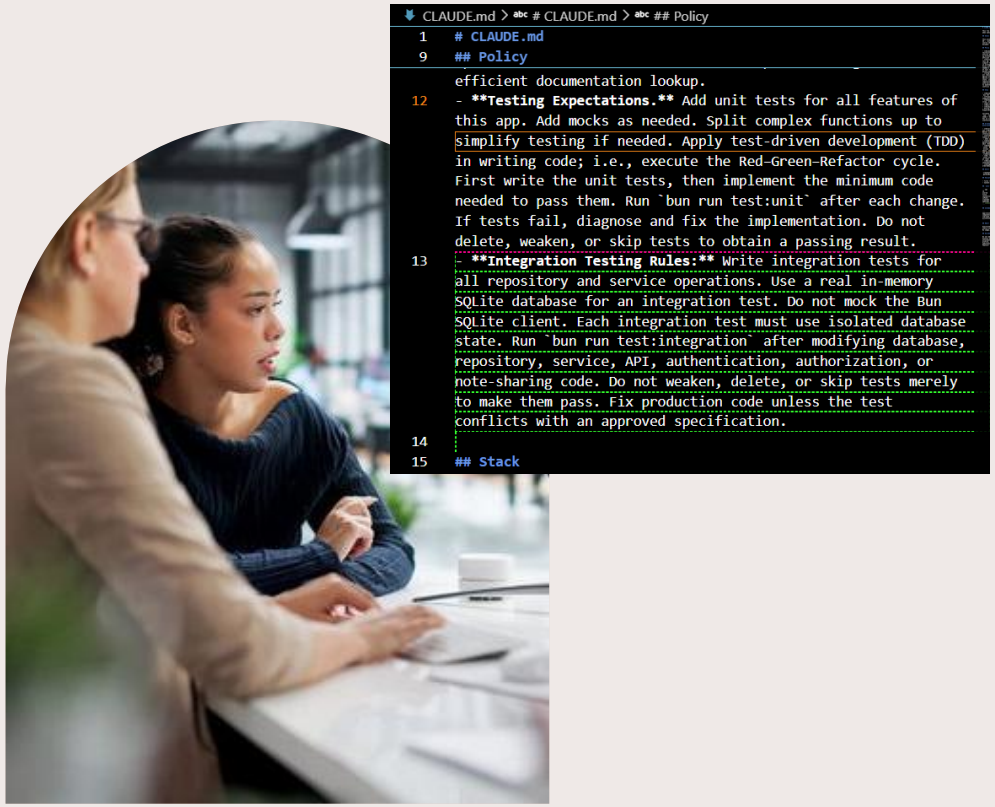
`bun run test:unit` passed.

- 3 tests passed
- 0 tests failed
- Ran in ~176ms across 1 file



- Add the **tests** folder to the current project.
- Add the **unit** subfolder in that folder.
- Add the **validation.test.ts** file for a unit test as shown on the left.
- Run **bun run test:unit**.

INTEGRATION TESTS



CLAUDE.md > # CLAUDE.md > ## Policy

```
1   # CLAUDE.md
9   ## Policy
    efficient documentation lookup.
12  - **Testing Expectations.** Add unit tests for all features of
    this app. Add mocks as needed. Split complex functions up to
    simplify testing if needed. Apply test-driven development (TDD)
    in writing code; i.e., execute the Red-Green-Refactor cycle.
    First write the unit tests, then implement the minimum code
    needed to pass them. Run `bun run test:unit` after each change.
    If tests fail, diagnose and fix the implementation. Do not
    delete, weaken, or skip tests to obtain a passing result.
13  - **Integration Testing Rules:** Write integration tests for
    all repository and service operations. Use a real in-memory
    SQLite database for an integration test. Do not mock the Bun
    SQLite client. Each integration test must use isolated database
    state. Run `bun run test:integration` after modifying database,
    repository, service, API, authentication, authorization, or
    note-sharing code. Do not weaken, delete, or skip tests merely
    to make them pass. Fix production code unless the test
    conflicts with an approved specification.
14
15  ## Stack
```

- **Integration tests verify whether multiple real application components (such as application logic, repository layer, raw SQL, Bun SQLite client, SQLite database) work correctly together.**
- **Place the following governance rule in the project's CLAUDE.md:**
  - ****Testing Rules:** Write integration tests for all repository and service operations. Use a real in-memory SQLite database for an integration test. Do not mock the Bun SQLite client. Each integration test must use isolated database state. Run `bun run test:integration` after modifying database, repository, service, API, authentication, authorization, or note-sharing code. Do not weaken, delete, or skip tests merely to make them pass. Fix production code unless the test conflicts with an approved specification.**
  - **This is an instruction-level trigger. Claude reads the rule and decides whether to run the command. It is not fully deterministic enforcement. The agent might overlook the instruction, or misclassify a change, or prefer not to invoke the command.**



tests > integration > TS sqlite.integration.test.ts > ...

```ts
import { describe, expect, it } from 'bun:test';
import { Database } from 'bun:sqlite';

describe('sqlite integration', () => {
  it('writes and reads rows in an in-memory SQLite database', () => {
    const db = new Database(':memory:');

    db.exec(`
      CREATE TABLE notes (
        id INTEGER PRIMARY KEY,
        title TEXT NOT NULL
      );
    `);

    const insert = db.query('INSERT INTO notes (title) VALUES (?)');
    const row = insert.run('integration note');

    const selected = db
      .query('SELECT id, title FROM notes WHERE id = ?')
      .get(Number(row.lastInsertRowid)) as { id: number; title: string } | null;

    expect(selected).not.toBeNull();
    expect(selected?.title).toBe('integration note');

    db.close();
  });
});
```

Running the integration test command now and I'll return the exact pass/fail result.

Ran `bun run test:integration`

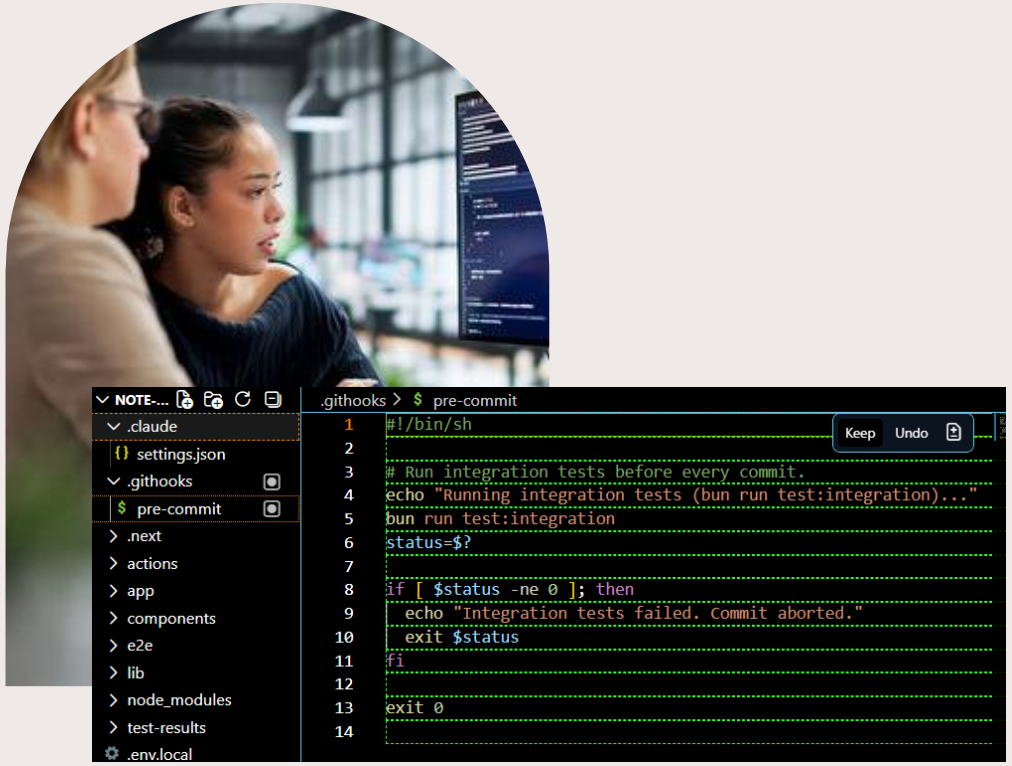
`bun run test:integration` passed.

- 1 test passed
- 0 tests failed
- Ran in ~104ms across 1 file

INTEGRATION TESTS

- Add the **integration** subfolder in the **tests** folder.
- Add the **sqlite.integration.test.ts** file for an integration test as shown on the left.
- Run **bun run test:integration**.

HOOKS FOR DETERMINISTIC INTEGRATION TESTS



- **To trigger unit and integration tests deterministically before every Git commit,**
  - **Add a repository-managed hook at .githooks/pre-commit.**
  - **Run git config core.hooksPath .githooks to configure Git to invoke .githooks/pre-commit every time anyone runs git commit.**
- **The workflow becomes: Claude modifies code → git commit → pre-commit hook runs → integration tests → Pass → commit allowed; Fail → commit blocked.**

CI FOR DETERMINISTIC INTEGRATION TESTS



.github > workflows > ! integration-tests.yml

```yaml
name: Integration Tests

on:
  pull_request:
    types: [opened, synchronize, reopened, ready_for_r

jobs:
  integration:
    if: github.event.pull_request.draft == false
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Bun
        uses: oven-sh/setup-bun@v2
        with:
          bun-version: latest

      - name: Install dependencies
        run: bun install --frozen-lockfile

      - name: Run integration tests
        run: bun run test:integration

```

- To trigger unit and integration tests deterministically in CI(Continuous Integration),
  - Create the **.github/workflows** folder in the current project.
  - Create the **integration-tests.yml** file in that folder as shown on the left.
- The workflow becomes: Claude Code creates a branch → Claude commits and pushes → Pull request is opened → GitHub Actions starts → Unit tests run → Integration tests run → Pass → PR may be merged; Fail → PR is blocked.
  - By using this GitHub Actions workflow, integration tests always run in CI on every PR.
- The next course is on CI/CD for Deterministic Verifications for Deploy, including format, lint, type check, unit tests, integration tests, E2E tests, production build, and deploy.

END-TO-END (E2E) UI TEST

note-taking > TS playwright.config.ts > ...

```ts
import { defineConfig, devices } from '@playwright/test';

const PORT = 3100;
const BASE_URL = `http://localhost:${PORT}`;

// Isolated test database so e2e never touches the dev `notes.d
// db.ts reads DATABASE_PATH; BETTER_AUTH_URL must match the se
// otherwise Better Auth's origin check rejects the sign-in POS
const TEST_ENV = {
  DATABASE_PATH: 'test.db',
  BETTER_AUTH_URL: BASE_URL,
};

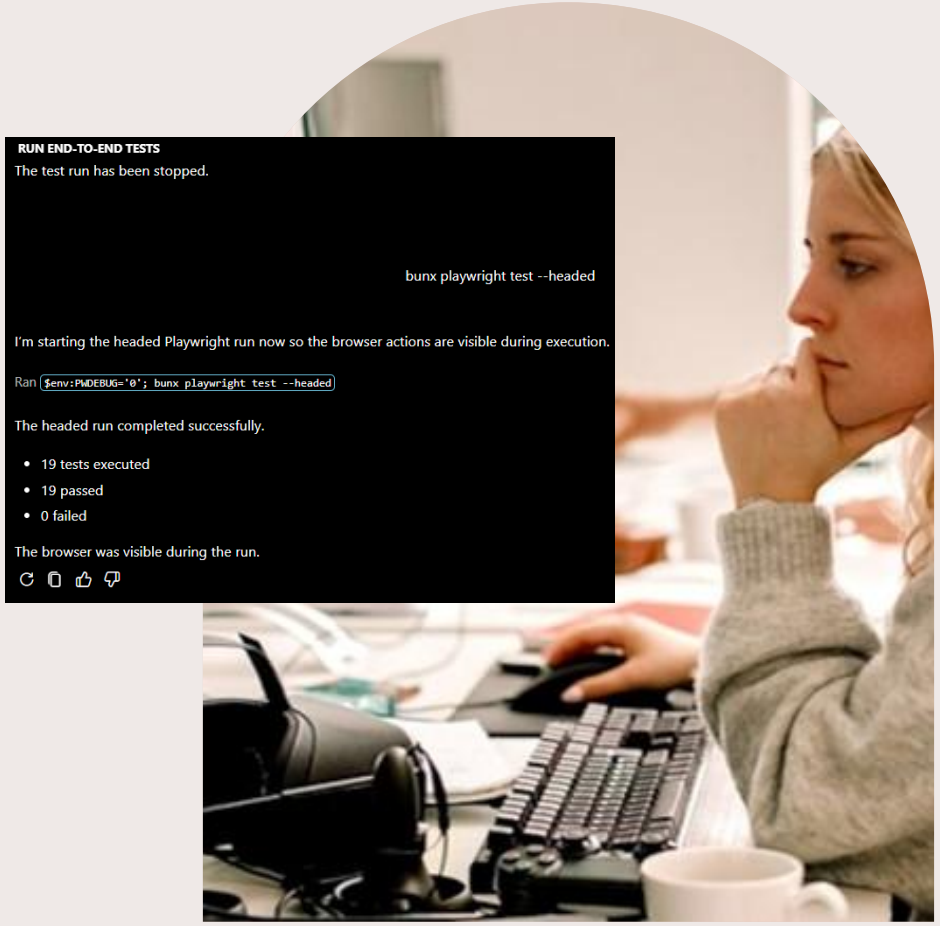
export default defineConfig({
  testDir: './e2e',
  globalSetup: './e2e/global-setup.ts',
  timeout: 60_000,
  expect: { timeout: 15_000 },
  fullyParallel: false,
  workers: 1, // one shared server + SQLite file; keep writes s
  forbidOnly: !!process.env.CI,
  reporter: [['list']],
  use: {
    baseURL: BASE_URL,
    trace: 'on-first-retry',
  },
  projects: [{ name: 'chromium', use: { ...devices['Desktop Chr
  webServer: {
    command: `bun --bun next dev --port ${PORT}`,
    url: BASE_URL,
    reuseExistingServer: false,
    timeout: 120_000,
    env: TEST_ENV,
  },
});

```



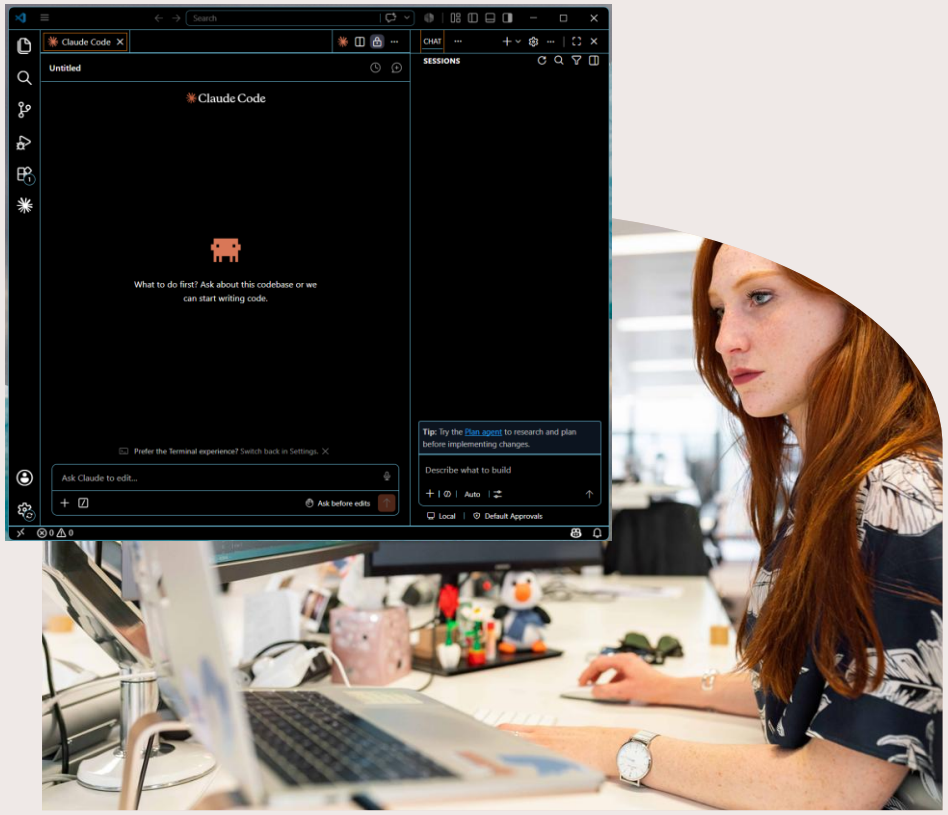
- An end-to-end (E2E) UI test is an automated test that controls your web application through a real browser, simulating what a user does and verifying what the user sees.
- It tests the application from the UI all the way through the backend and database.
- E2E tests typically cover representative user journeys, not every logical branch or error condition.
- You can use **Playwright** for E2E tests. It can launch a web browser, open web pages, click buttons and links, fill in forms, select UI elements, upload files, verify text and page state, test authentication flows, take screenshots, and run tests in parallel.
- Install Playwright in the **note-taking** project if it is not installed yet.
- Download **Chromium**.
- Install the **Playwright Test for VSCode** extension.
- Create the **playwright.config.ts** file as shown on the left.

END-TO-END (E2E) UI TEST



- Open the **note-taking** folder in the VSCode extension so that VSCode reloads with **note-taking** as the workspace root and so the Claude Code panel treats it as the project.
- Run **/clear**.
- Run **bun run test:e2e** if you want to run the test headlessly.
- Run **bunx playwright test --headed** if you want to see the browser live during the test.
- Run **PWDEBUG=1 bunx playwright test --headed** if you want to step through each action.

QUICK REFERENCE



- bun run test:unit => Run the unit test command defined in package.json.
- bun run test:integration => Run the integration test command defined in package.json.
- git config core.hooksPath .githooks => Configure Git to invoke **.githooks/pre-commit** every time anyone runs **git commit**.
- playwright.config.ts => Configure the Playwright E2E test automation
- bun run test:e2e => Run the E2E test command defined in package.json headlessly.
- bunx playwright test --headed => Run the E2E test, seeing the browser live during the test.
- .github/workflows/integration-tests.yml => Configure GitHub Actions workflow for Continuous Integration (CI)

THANK YOU

Dr. June Sung Park

june@kosta-online.com

www.kosta-online.com